

Chapter 3: Problem Solving and Search (Exam-Focused Notes)

Notation and Complexity Conventions

Symbols

- b : maximum branching factor (maximum number of successors of any node).
- d : depth of the shallowest goal (for unit step costs) / depth of the least-cost solution in many standard analyses.
- m : maximum depth of the state space (may be infinite).
- l : depth limit (Depth-Limited Search).
- $g(n)$: path cost from the start node to node n .
- C^* : optimal solution cost (minimum path cost to reach a goal).
- ε : minimum step cost, i.e., all step costs satisfy $c(x, a, y) \geq \varepsilon$ and $\varepsilon > 0$.

What “Time” and “Space” Mean

- **Time complexity**: number of nodes **generated** and/or **expanded** in the worst case.
 - **Generated**: created and inserted into the frontier.
 - **Expanded**: removed from the frontier and had successors generated.
- **Space complexity**: maximum number of nodes stored in memory at any time (frontier plus, for graph search, the closed set).

Geometric Series Toolkit

Nodes at depth i are at most b^i . Total nodes from depth 0 to depth k is

$$\sum_{i=0}^k b^i = 1 + b + b^2 + \cdots + b^k.$$

For $b > 1$, this sum equals

$$\sum_{i=0}^k b^i = \frac{b^{k+1} - 1}{b - 1}.$$

In Big- O terms, the sum is dominated by the largest term b^k , so

$$1 + b + \cdots + b^k = O(b^k).$$

1 Problem-Solving Agents

1.1 Simple Problem-Solving Agent (Offline Planning)

Listing 1: Pseudocode: SIMPLE-PROBLEM-SOLVING-AGENT

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  static: seq, an action sequence, initially empty
  state, some description of the current world state
  goal, a goal, initially null
  problem, a problem formulation

  state <- UPDATE-STATE(state, percept)
  if seq is empty then
    goal <- FORMULATE-GOAL(state)
    problem <- FORMULATE-PROBLEM(state, goal)
    seq <- SEARCH(problem)
  action <- FIRST(seq)
  seq <- REST(seq)
  return action
```

Key Ideas

- **Offline problem solving:** compute a complete plan before executing actions.
- **Online problem solving:** interleave planning and acting.

Self-Check

1. Why can offline planning fail in a dynamic environment?
2. What triggers re-planning in the algorithm?
3. Where does the computational burden occur in offline search?

2 Problem Types

Problem Type	Environment Properties	Solution Form
Single-state	Deterministic, fully observable	Fixed sequence of actions
Sensorless (conformant)	No perceptual information	Action sequence that works for all possible states
Contingency	Nondeterministic and/or partially observable	Conditional plan (if-then) / policy
Exploration	Unknown state space	Interleaved planning and acting

Self-Check

1. Why does a contingency problem require branching?
2. What is a belief state in sensorless problems?

3. Why can exploration not be purely offline?

3 Single-State Problem Formulation

A single-state problem is defined by four components:

1. **Initial state.**
2. **Successor function** $S(x)$ returning pairs (a, y) where a is an action and y is the result state.
3. **Goal test.**
4. **Path cost**, additive sum of step costs. Step costs satisfy $c(x, a, y) \geq 0$.

Examples

Route Planning (Romania-style).

- States: cities.
- Actions: drive between connected cities.
- Goal test: be in the target city.
- Path cost: sum of distances.

Vacuum World.

- States: (robot location, dirt configuration).
- Actions: Left, Right, Suck, NoOp.
- Goal test: no dirt remains.
- Path cost: 1 per action (and 0 for NoOp if defined that way).

8-Puzzle.

- States: tile configurations.
- Actions: move blank up/down/left/right when legal.
- Goal test: equals the goal configuration.
- Path cost: 1 per move.

State vs Node

Concept	Meaning
State	A world configuration; does not include search bookkeeping.
Node	A search-tree record: (state, parent, action, depth, path cost $g(n)$).

4 General Tree Search Template

Listing 2: Pseudocode: TREE-SEARCH

```
function TREE-SEARCH(problem, fringe) returns solution or failure
  fringe <- INSERT(MAKE-NODE(INITIAL-STATE(problem)), fringe)
  loop:
    if fringe is empty then return failure
    node <- REMOVE-FRONT(fringe)
    if GOAL-TEST(STATE(node)) then return SOLUTION(node)
    fringe <- INSERTALL(EXPAND(node, problem), fringe)
```

A **strategy** is determined by how the frontier (fringe) is ordered:

- FIFO queue: Breadth-First Search (BFS).
- LIFO stack: Depth-First Search (DFS).
- Priority queue by increasing $g(n)$: Uniform-Cost Search (UCS).

5 Breadth-First Search (BFS)

Strategy

Expand the shallowest unexpanded node first. Frontier is a FIFO queue.

Dry Run (Queue Trace)

Assume $A \rightarrow \{B, C\}$ and $B \rightarrow \{D, E\}$.

Step	Queue Before	Expanded	Children Added	Queue After
0	[A]	—	—	[A]
1	[A]	A	B, C	[B, C]
2	[B, C]	B	D, E	[C, D, E]

Completeness and Optimality

- **Complete:** Yes, if b is finite.
- **Optimal:** Yes, if all step costs are equal (unit-cost setting).

Time Complexity: $O(b^{d+1})$ (Full Derivation)

Let the shallowest goal be at depth d .

1. Nodes at depth i are at most b^i .
2. BFS expands nodes by depth. To reach depth d , it must expand all nodes at depths $0, 1, \dots, d-1$ and may need to expand many nodes at depth d in the worst case.

3. The number of nodes up to depth d is

$$\sum_{i=0}^d b^i = O(b^d).$$

4. Additionally, when expanding nodes at depth d , BFS generates their children at depth $d + 1$. There are at most b^d nodes at depth d , each generating up to b successors, so generated nodes at depth $d + 1$ can be as large as

$$b \cdot b^d = b^{d+1}.$$

5. Therefore, a standard worst-case upper bound on the number of generated nodes is

$$O(b^d) + O(b^{d+1}) = O(b^{d+1}).$$

$$\boxed{T_{\text{BFS}} = O(b^{d+1})}$$

Space Complexity: $O(b^{d+1})$ (Full Derivation)

BFS stores the frontier. Near the deepest explored layer, the frontier can contain on the order of b^{d+1} nodes (the next level generated from depth d). Hence

$$\boxed{S_{\text{BFS}} = O(b^{d+1})}$$

Watch Out

- Do not claim BFS is optimal without stating the equal step-cost condition.
- BFS often fails in practice due to memory, not CPU time, because the frontier grows exponentially.

Visual Sketching Guide

Sketch a tree with horizontal layers. Mark BFS as expanding layer 0, then layer 1, then layer 2, etc. Draw a large queue holding an entire layer to emphasize memory growth.

Self-Check

1. Why must BFS expand all nodes at depths $< d$ before any node at depth d ?
2. Where does the extra factor b come from in the $O(b^{d+1})$ bound?
3. Why is BFS not optimal when step costs are unequal?

6 Uniform-Cost Search (UCS)

Strategy

Expand the node with smallest $g(n)$. Frontier is a priority queue ordered by increasing $g(n)$.

Completeness and Optimality

- **Complete:** Yes, if $\varepsilon > 0$.
- **Optimal:** Yes.

Time Complexity: $O(b^{C^*/\varepsilon})$ (Full Derivation)

1. UCS expands nodes in increasing path cost $g(n)$.
2. Let C^* be the optimal solution cost. UCS will expand all nodes with $g(n) < C^*$ and possibly some with $g(n) = C^*$ before the optimal goal is selected.
3. If every step cost is at least $\varepsilon > 0$, then any path of k steps has cost at least $k\varepsilon$:

$$g(n) \geq k\varepsilon.$$

4. For a node to satisfy $g(n) \leq C^*$, we must have

$$k\varepsilon \leq C^* \quad \Rightarrow \quad k \leq \frac{C^*}{\varepsilon}.$$

5. Thus, UCS may need to consider nodes up to depth

$$k_{\max} = \left\lfloor \frac{C^*}{\varepsilon} \right\rfloor.$$

6. The number of nodes up to depth $k_{\max}$ is

$$\sum_{i=0}^{k_{\max}} b^i = O(b^{k_{\max}}).$$

7. Substitute $k_{\max} \approx \frac{C^*}{\varepsilon}$:

$$T_{\text{UCS}} = O\left(b^{C^*/\varepsilon}\right).$$

Space Complexity: $O(b^{C^*/\varepsilon})$ (Full Derivation)

UCS stores the frontier containing nodes with cost up to C^* . The number of such nodes is bounded by the same exponential node-count argument, hence

$$S_{\text{UCS}} = O\left(b^{C^*/\varepsilon}\right).$$

Watch Out

- Do not forget the condition $\varepsilon > 0$ for completeness.
- UCS expands by *cost*, not by depth.

Visual Sketching Guide

Sketch nodes labeled by $g(n)$ values and show expansion as increasing-cost “contours” rather than depth layers.

Self-Check

1. Why is $\varepsilon > 0$ required?
2. When does UCS behave exactly like BFS?
3. Can UCS expand a deeper node before a shallower node? Give a cost-based reason.

7 Depth-First Search (DFS)

Strategy

Expand the deepest unexpanded node first. Frontier is a LIFO stack.

Completeness and Optimality

- **Complete:** No (fails in infinite-depth spaces and with cycles under tree search).
- **Optimal:** No.

Time Complexity: $O(b^m)$ (Full Derivation)

1. In the worst case, DFS explores the entire search tree down to maximum depth m .
2. The number of nodes in a full b -ary tree of depth m is

$$\sum_{i=0}^m b^i = O(b^m).$$

$$\boxed{T_{\text{DFS}} = O(b^m)}.$$

Space Complexity: $O(bm)$ (Full Derivation)

1. DFS stores the current path from root to a leaf, which has length at most m .
2. At each level, it may need to remember up to b successor options (unexplored siblings or iterators).
3. Therefore, memory scales like m levels times b branching:

$$\boxed{S_{\text{DFS}} = O(bm)}.$$

Watch Out

- DFS can get stuck exploring an infinite branch and never return.
- DFS is not optimal even with unit step costs, because it may find a deeper goal first.

Visual Sketching Guide

Draw a tree with a single highlighted deep path. Show a stack containing nodes along this path plus a few pending siblings.

Self-Check

1. Why is DFS memory much smaller than BFS memory?
2. Why can DFS fail to terminate even if a solution exists elsewhere?
3. What modification prevents repeated-state looping in cyclic graphs?

8 Depth-Limited Search (DLS)

Strategy

DFS with a maximum depth limit l . The algorithm can return **solution**, **failure**, or **cutoff**.

Time Complexity: $O(b^l)$ (Full Derivation)

DLS explores all nodes up to depth l in the worst case:

$$\sum_{i=0}^l b^i = O(b^l).$$

$$T_{\text{DLS}} = O(b^l).$$

Space Complexity: $O(bl)$ (Full Derivation)

DFS-style storage with maximum depth l :

$$S_{\text{DLS}} = O(bl).$$

Completeness

Complete if $l \geq d$ (assuming finite branching and no infinite loops along a path).

Self-Check

1. What does “cutoff” mean and why is it necessary?
2. If $l < d$, what outcome should DLS return and why?
3. Why is DLS not optimal even with unit costs?

9 Iterative Deepening Search (IDS)

Strategy

Repeatedly run DLS with limits $0, 1, 2, \dots$ until a solution is found.

Completeness and Optimality

- **Complete:** Yes (finite b).
- **Optimal:** Yes if all step costs are equal.

Time Complexity: $O(b^d)$ (Full Derivation)

1. Nodes at depth i are expanded once for each limit value from i to d .
2. Therefore, nodes at depth i are expanded $(d - i + 1)$ times.
3. Nodes at depth i are at most b^i .
4. Total expansions are bounded by:

$$\sum_{i=0}^d (d - i + 1)b^i = (d + 1)b^0 + db^1 + (d - 1)b^2 + \cdots + 1 \cdot b^d.$$

5. The dominant term is b^d . The earlier terms are smaller by factors of b (up to multiplicative constants involving d).
6. Hence the sum is $O(b^d)$.

$$\boxed{T_{\text{IDS}} = O(b^d)}.$$

Space Complexity: $O(bd)$ (Full Derivation)

Each iteration uses DFS-style space up to depth d , so

$$\boxed{S_{\text{IDS}} = O(bd)}.$$

Watch Out

- IDS repeats work at shallow levels, but the total cost is dominated by the deepest level.
- IDS is optimal only in unit-cost settings.

Visual Sketching Guide

Draw the same tree multiple times with increasing horizontal cutoff lines at depths $0, 1, 2, \dots$, showing that each iteration goes one level deeper.

Self-Check

1. Why does repeated work not dominate IDS time?
2. Why is IDS memory much smaller than BFS memory?
3. Under what condition does IDS return an optimal solution?

10 Repeated States and Graph Search

Graph Search with a Closed Set

Listing 3: Pseudocode: GRAPH-SEARCH

```
function GRAPH-SEARCH(problem, fringe) returns a solution, or failure
  closed <- empty set
  fringe <- INSERT(MAKE-NODE(INITIAL-STATE(problem)), fringe)
  loop:
    if fringe is empty then return failure
    node <- REMOVE-FRONT(fringe)
    if GOAL-TEST(STATE(node)) then return node
    if STATE(node) not in closed then
      add STATE(node) to closed
      fringe <- INSERTALL(EXPAND(node, problem), fringe)
```

Key Effects

- Prevents expanding the same state repeatedly (reduces redundant work).
- Requires extra memory for the closed set.
- Using a hash table for closed membership gives average $O(1)$ lookup.

Complexity Notes

- Time can drop significantly in graphs with many cycles or multiple paths to the same state.
- Worst-case space can still be exponential in d if the number of reachable states grows like $O(b^d)$.

Self-Check

1. Why does a closed set help DFS terminate on cyclic graphs?
2. Why can graph search still require exponential memory in the worst case?
3. What data structure should be used for closed and why?

11 Decision Comparison Table

Algorithm	Complete	Optimal	Time	Space
BFS	Yes	Yes (unit costs)	$O(b^{d+1})$	$O(b^{d+1})$
UCS	Yes ($\varepsilon > 0$)	Yes	$O(b^{C^*/\varepsilon})$	same
DFS	No	No	$O(b^m)$	$O(bm)$
DLS	Yes if $l \geq d$	No	$O(b^l)$	$O(bl)$
IDS	Yes	Yes (unit costs)	$O(b^d)$	$O(bd)$

12 Complexity Cheat Sheet

Algorithm	Time	Space	Key Conditions
BFS	$O(b^{d+1})$	$O(b^{d+1})$	Optimal only if equal step costs
UCS	$O(b^{C^*/\varepsilon})$	same	Requires $\varepsilon > 0$ for completeness
DFS	$O(b^m)$	$O(bm)$	Not complete in infinite/cyclic tree search
DLS	$O(b^l)$	$O(bl)$	Complete if $l \geq d$
IDS	$O(b^d)$	$O(bd)$	Optimal only if equal step costs